



SUMMER PROJECT REPORT
ON
DEVELOPMENT OF
“NETWORK MONITORING TOOL”

NAME OF STUDENT: Utkarsh Dixit

Id: 2015UEC1490

INSTITUTE – Malaviya National Institute of Technology, Jaipur

HOST INSTITUTE – Indian Institute Of Technology, Kanpur

CERTIFICATE

Issue Date: 27-June-2017

To Whom So Ever It May Concern

This is to certify that project entitled Network Monitoring Tool, designed on LAMP architecture is an original work carried out by Bachelor of Technology (4th semester) student Mr. **UTKARSH DIXIT**, of Malaviya National Institute of Technology, Jaipur under my supervision at **Indian Institute of Technology, Kanpur**.

The matter embodied in this project is a genuine work done by the student and has not been submitted to the university or to any other university/institute for the fulfillment of the requirement of any course of study.

Hope you have found all the above details in order.



(Mr. Navpreet Singh)
Chief Computer Engineer
Computer Centre
Indian Institute of Technology, Kanpur

Acknowledgement

I would like to thank my mentor; **Mr. Navpreet Singh (Chief engineer, Computer center, I.I.T. Kanpur)** for giving me a wonderful opportunity to work at his prestigious institution .I would like to take this chance to express my gratitude towards him for providing me a chance to interact with technology and grasp an understanding of networking concepts and also into its practical applications. Needless to say this undertaking would not have been possible without his guidance.

Also I would like to take a moment to express my sincere thanks towards **Mr. Saurabh Malhotra (I.I.T. Kanpur)** whose technical guidance on each and every step during my project is invaluable. He never did falter and gave me the best technical guidance which reflects in the project. I got a better understanding of how networks actually exist and are maintained in the real world.

Under them I had a chance to be introduced to the world of networking, looked into the industry-scale applications and arrangements of devices and even had a wonderful opportunity to look into the widespread domains of networking.

Once again I express my heartfelt gratitude towards all of them and needless to say that their contribution cannot be quantified in any way imaginable.

UTKARSH DIXIT

(Project Trainee, I.I.T. Kanpur)

INDEX

<u>Title</u>	<u>Page No.</u>
1. Certificate.....	2
2. Acknowledgement.....	3
3. Abstract.....	5
4. Objective.....	6
5. Introduction.....	7
6. Methodology.....	9
7. Visuals.....	11
8. Softwares and tools used.....	15
9. Results and Application.....	18
10. Future Aspects.....	19
11. Conclusion.....	20
• Appendix - A	
• Appendix - B	

Abstract

In today's world, technology is at its pinnacle. Such a great growth has been possible with a lot of factors working endlessly like cogs in a machine and has changed innumerable lives. Networking, if we must say is one of the major factors which have made all of this possible and we reap the benefits in our daily life.

Today everything is within our reach with the touch of a button. Whether it's shopping online or transferring money or talking to people face to face sitting several thousand kilometers away, everything is possible and networking is what has made it possible.

So, keeping in mind the numerous aspects of a networking domain, this project focusses on analyzing this network and networking technology.

In this particular literature on networking, we aim at designing a network monitoring tool. It provides a step by step methodology, reasons and graphical illustrations as to what is being done and why such steps are taken.

As a prologue as to what topics might be uncovered are; LAMP Architecture, html login forms, Shell scripting and many more. Also the command line codes at each individual step have been included to make the understanding process a little more refined.

In conclusion, this project report aims at describing the networking concepts that have been learned and applied; the outcomes that are expected using the tools provided and the final objective of undertaking this particular project.

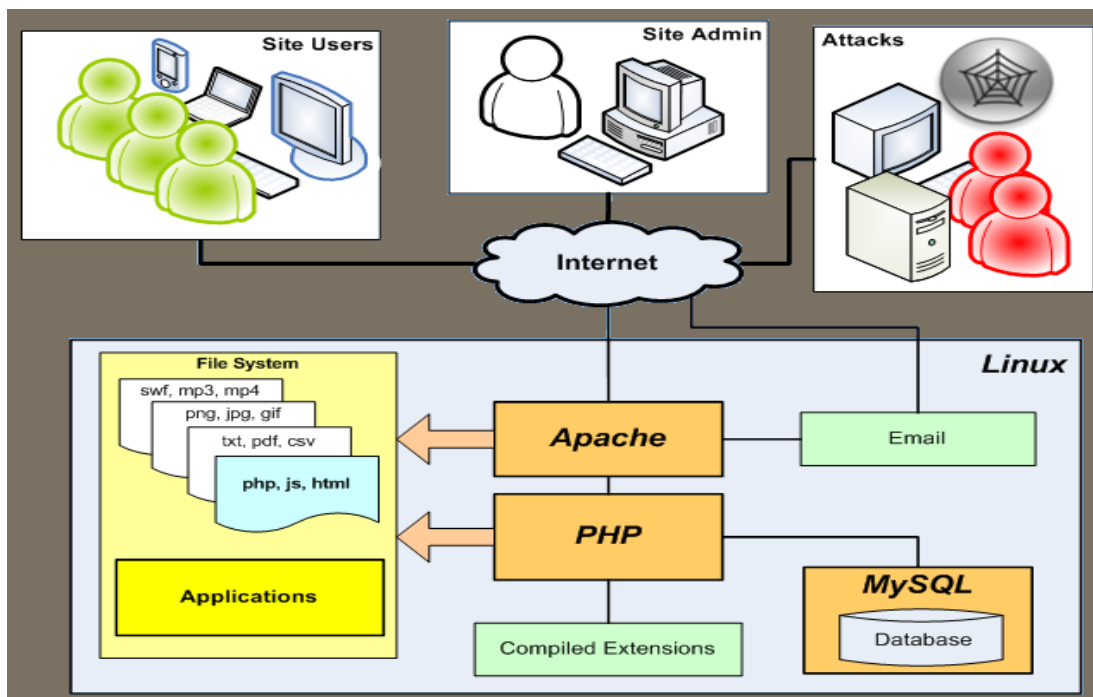


OBJECTIVE

The objective of this project is to develop a network monitoring tool. A network monitoring tool oversees the operations of a network. Such a tool is used to ensure availability and overall performance of computers (hosts) and network services. It monitors the network for the problems caused by overloaded and/or crashed servers, network connections and other devices. It notifies the network administrator in case of outages so that he can handle the problem immediately.

This Network Monitoring Tool essentially detects abnormal behaviour of any host and in case of such abnormality checks for its activities by decrypting data (packets) sent across the network by this host machine. Thus, by reading this packet information we can analyse the nature of its conversation. Then we can take necessary action against that host in order to make it function properly.

Network Monitoring Tool (NMT) captures the packets using “**tcpdump**”. Tcpdump can be used to capture some or all packets received by a network interface. The range of packets captured can be specified by the using a combination of logical operators and parameters such as source and destination Mac or IP addresses, protocol types (IP and Ethernet) and TCP/UDP port numbers.



INTRODUCTION

Network monitoring for a network is a critical IT function that can save money in network performance, employee productivity and infrastructure cost overruns. Network monitoring can be achieved using various software tools or a combination of plug-and-play hardware and software appliance solutions.

A network monitoring tool monitors an internal network for problems. It can find and help resolve snail-paced webpage downloads, lost-in-space e-mails, questionable user activity and file delivery caused by overloaded/crashed servers as well as dicey network connections or other devices.

When failures, unacceptably slow response or other unexpected behaviour is detected these tools send messages to designate locations to notify system administrators, thus providing more operational visibility. Virtually any kind of network can be monitored. It doesn't matter whether it's wireless or wired, a corporate LAN, VPN (Virtual Private Network) or service provider WAN. We can monitor devices on different operating systems with a multitude of functions, ranging from servers, routers and switches to cell-phones.

The Network Monitoring Tool that I have designed is a simple one but a very effective tool. Different monitoring tools are designed using different architectures. My Network Monitoring Tools is based on the **LAMP** architecture, where:

- 'L' stands for Linux (the operating system).
- 'A' stands for Apache (the web server).
- 'M' stands for the MySQL (the Database Management System).
- 'P' stands for Perl/PHP/Python (the server-side scripting languages).

As stated earlier, in order to design a network monitoring tool the first and the most important step is to capture all the data sent across the network.

Any data that is sent across the network is sent in the form of data units that are called **packets**. It turns out that everything one does on the Internet involves packets. For example, every Web page that we receive comes as a series of packets, and every e-mail we send leaves as a series of packets.

Networks that ship data around in small packets are called **packet switched networks**.

Each packet carries the information that will help it in getting to its destination i.e. the sender's IP address, the intended receiver's IP address, something that tells the network how many packets this e-mail message has been broken into and the number of this particular packet.

The packets carry the data in the protocols that the Internet uses:

Transmission Control Protocol/Internet Protocol (TCP/IP). Each packet contains part of the body of your message. A typical packet contains perhaps 1,000 or 1,500 bytes.

Most packets splits into three parts:

- 1) **Header** contains instructions about the data carried by the packet. For e.g. length of the packet, packet number, protocol, destination address etc.
- 2) **Payload** also called the **body** or **data** of a packet. This is the actual data that the packet is delivering to the destination. If a packet is fixed-length, then the payload may be **padded** with blank information to make it of the right size.
- 3) **Trailer** sometimes called the **footer**, typically contains a couple of bits that tell the receiving device that it has reached the end of the packet.

In IITK, all the application terminal are connected via LAN. To keep the track record of all the communication and all the data transfer taking place the Institute needs various network tools. Most common and of course very useful tools are “**NETWORK MONITORING TOOLS**”.

The program has been designed for three protocols:

- TCP (Transmission Control Protocol)
- UDP (User Datagram Protocol)
- ARP (Address Resolution Protocol)

METHODOLOGY

To make this project working following steps have been used in sequence:

1. **PACKET CAPTURING**: First and the most basic step is sniffing packets passing through the network. For this we have used a command-line packet capture tool '**tcpdump**' which is supported by UNIX/LINUX/fedora. However on windows we can run 'WINDUMP'.

➤ **TCPDUMP** can provide very detailed information about any network conversation that runs across the wires.

➤ An advantage of TCPDUMP is that it can be run remotely through a SSH or TELNET Session, and the machine running TCPDUMP does not have to be running x-windows. The application uses very little overhead, since it's a non-graphical interface.

➤ Running tcpdump command on terminal without any of its option will Show something as screenshot below:

➤ To stop capturing packets, press ^C (ctrl+C) or ^Z (Ctrl+Z) or ^D (Ctrl+D)

➤ When stopped it will show something like:

```
1194 packets captured
19469 packets received by filter
18106 packets dropped by kernel
```

2. **PACKET PROCESING**: The second step is to process the captured packets so that they can be written in the database. Hence before moving further we shall process the packets by writing a shell script on the **vim editor**.

For processing, following UNIX commands have been used:

- **Terminal – script.sh:** open terminal for writing shell script. Now if required to write a plain script to be run on terminal it starts with **#!/bin/sh**

Before executing we will be required to change file permission using 'chmod'. Then to execute write './network.sh' on terminal (assuming file name to be network.sh)

- **grep:** (searching for a pattern) to search for all IP packets and put them into a separate file.
- **Cut:** (slitting a file vertically) to appropriately cut the data into file so that they can be written into database.
- **Paste:** (pasting files) to paste all cut files to one so that they can be loaded directly to database.
- **rm:** (permanently deletes a file) to remove all the files when the user logs out for privacy purpose.
- **:set nu:** displays line number on a text file.

3. **DATA STORAGE:** After processing, the data needs to be stored into MySQL database.

It does so by using the following SQL commands.

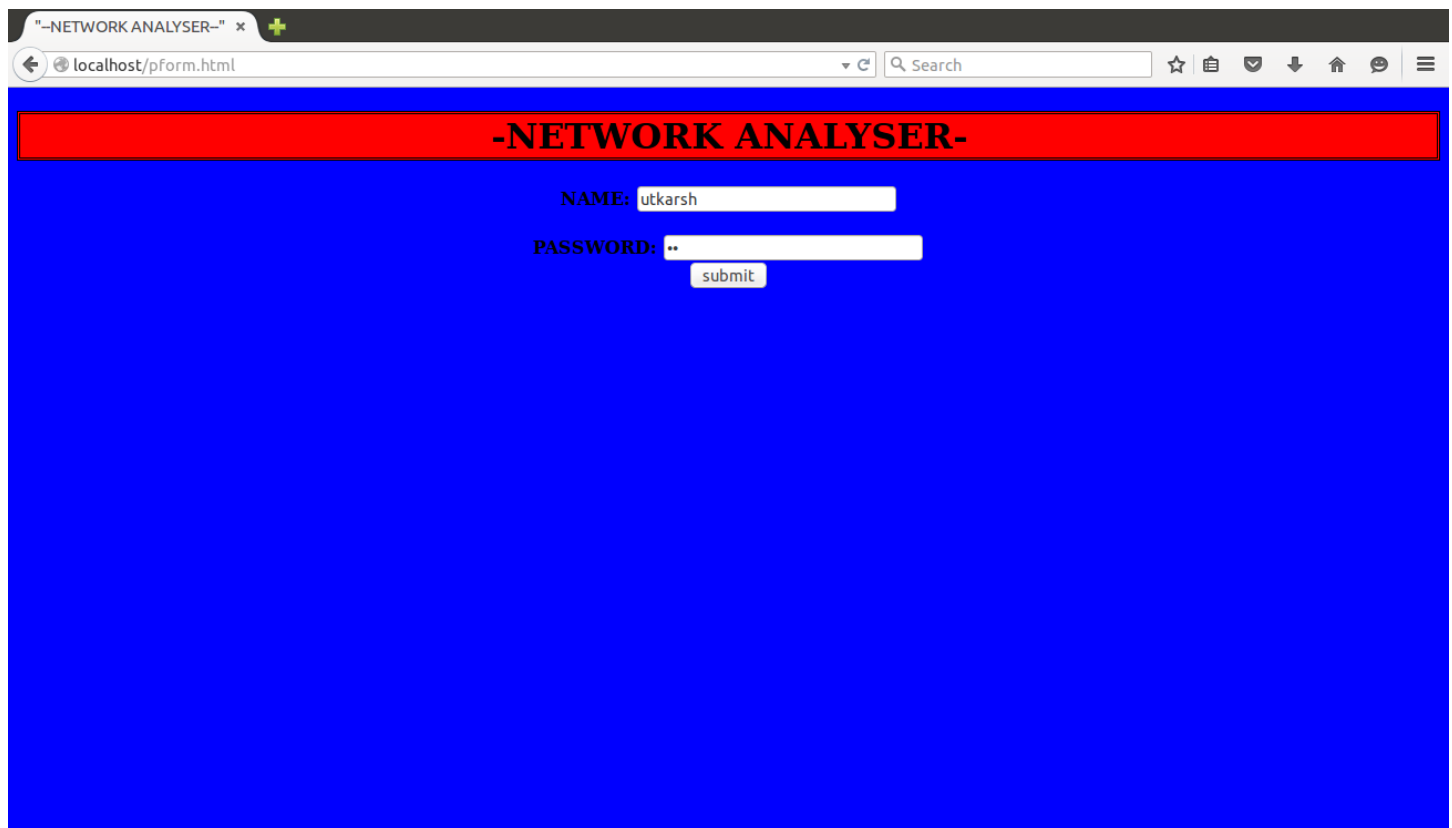
- ✓ CREATE DATABASE <database name> (to create the database)
- ✓ USE <database name> //(to use the database)
- ✓ CREATE TABLE <table name><(table attributes)> (to create any table in database)
- ✓ LOAD DATA LOCAL INFILE<filename> INTO TABLE <table name> (to load the data of a file into a table in the database)

Note: Clearly, the database is created first and then the table is created and then the file is loaded to that table.

4. **USER INTERFACE:** This is the next and very essential part of the monitoring tool. It is implemented using PHP.

The first page is the login page: From this page user can go to the search page by logging into their accounts.

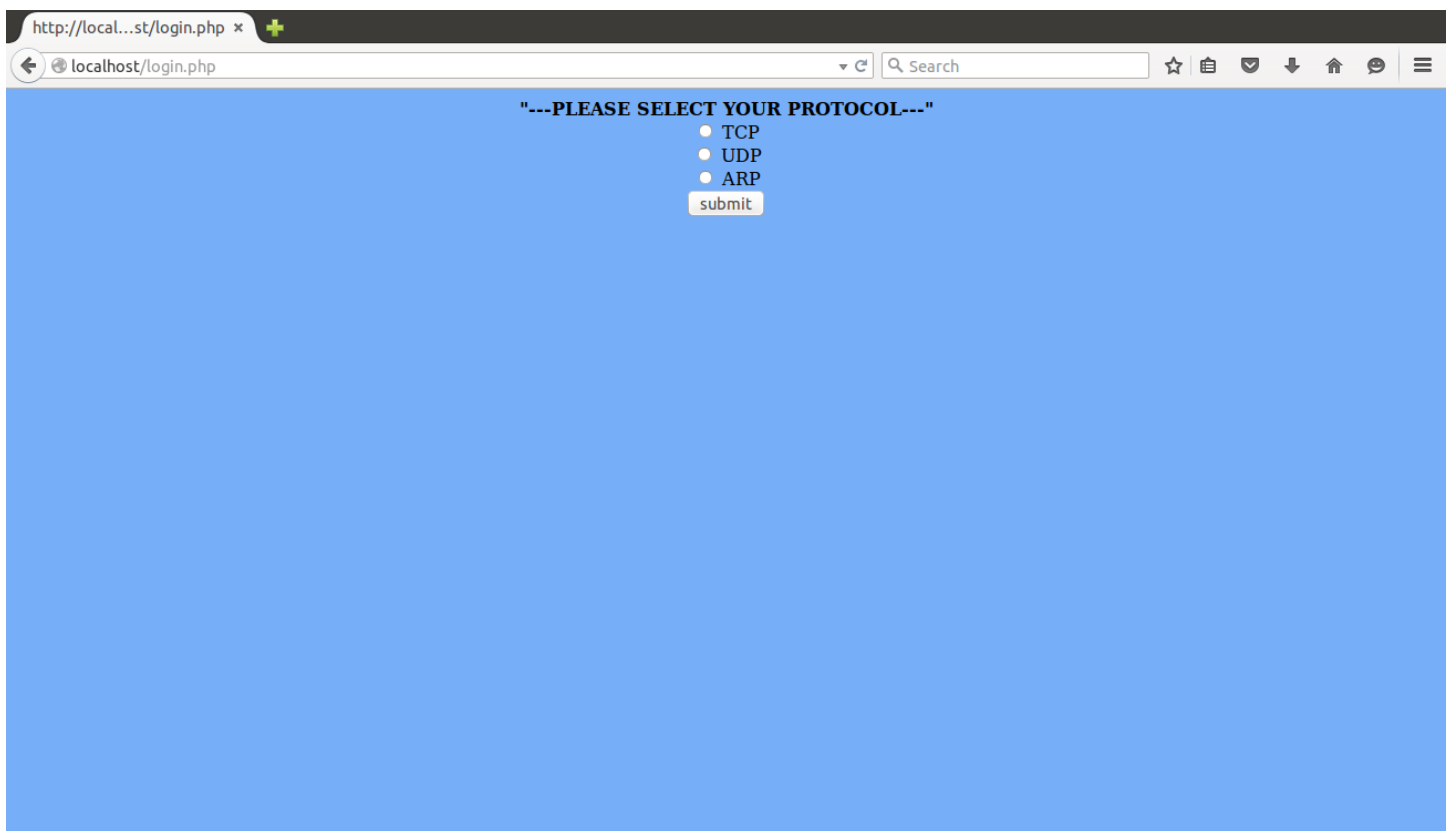
LOGIN PAGE:



The screenshot shows a web browser window with the title "-NETWORK ANALYSER-". The address bar shows "localhost/pform.html". The page has a blue background. At the top, there is a red header bar with the text "-NETWORK ANALYSER-". Below the header, there is a login form with two input fields: "NAME:" with the value "utkarsh" and "PASSWORD:" with two asterisks. A "submit" button is located below the password field.

- ✓ Provides a username field to input name of the user.
- ✓ Provides a password field to login.
- ✓ Form is submitted by clicking on the submit button.

If the credentials entered are correct then the following page will be displayed:



- ✓ The user selects the desired packet to be displayed in the next page.
- ✓ The fields are timestamp, source mac, destination mac, source IP, destination

IP, sender's length, receiver's length, source port and destination port for the ARP/IP/UDP packets.

On clicking submit button you will see the page below:

This page shows the details of each packet of TCP field as captured by tcpdump:

http://local...otocols.php x +

localhost/protocols.php Search

---TCP PACKETS---

TIMESTAMP	SOURCE IP	DESTINATION IP	SOURCE MAC	DESTINATION MAC	SOURCE PORT	DESTINATION PORT
18:39:52.067539	2405:204:a186:5477:9c51:9f19:9c40:255b	2404:6800:4009:805::200e	b8:76:3f:60:b3:c9	ac:c1:ee:67:6a:6f		
18:39:52.158070	2404:6800:4009:805::200e	2405:204:a186:5477:9c51:9f19:9c40:255b	ac:c1:ee:67:6a:6f	b8:76:3f:60:b3:c9		
18:39:52.899524	192.168.43.56	117.18.237.29	b8:76:3f:60:b3:c9	ac:c1:ee:67:6a:6f		
18:39:53.037593	117.18.237.29	192.168.43.56	ac:c1:ee:67:6a:6f	b8:76:3f:60:b3:c9		
18:40:00.419526	192.168.43.56	91.189.90.41	b8:76:3f:60:b3:c9	ac:c1:ee:67:6a:6f		
18:40:00.659433	91.189.90.41	192.168.43.56	ac:c1:ee:67:6a:6f	b8:76:3f:60:b3:c9		
18:40:01.062481			ac:c1:ee:67:6a:6f	33:33:ff:40:25:5b		
18:40:02.179528	2405:204:a186:5477:9c51:9f19:9c40:255b	2404:6800:4009:805::200e	b8:76:3f:60:b3:c9	ac:c1:ee:67:6a:6f	58585	80
18:40:02.286289	2404:6800:4009:805::200e	2405:204:a186:5477:9c51:9f19:9c40:255b	ac:c1:ee:67:6a:6f	b8:76:3f:60:b3:c9	80	58585
18:40:03.043525	192.168.43.56	117.18.237.29	b8:76:3f:60:b3:c9	ac:c1:ee:67:6a:6f	80	50502
18:40:03.157038	117.18.237.29	192.168.43.56	ac:c1:ee:67:6a:6f	b8:76:3f:60:b3:c9	50502	80
18:40:08.447932			ac:c1:ee:67:6a:6f	33:33:ff:60:b3:c9	57035	80
18:40:08.447957			b8:76:3f:60:b3:c9	ac:c1:ee:67:6a:6f	80	57035
18:40:10.659524	192.168.43.56	91.189.90.41	b8:76:3f:60:b3:c9	ac:c1:ee:67:6a:6f	58585	80
18:40:10.900698	91.189.90.41	192.168.43.56	ac:c1:ee:67:6a:6f	b8:76:3f:60:b3:c9	80	58585
18:40:12.291535	2405:204:a186:5477:9c51:9f19:9c40:255b	2404:6800:4009:805::200e	b8:76:3f:60:b3:c9	ac:c1:ee:67:6a:6f	50502	80
18:40:12.426083	2404:6800:4009:805::200e	2405:204:a186:5477:9c51:9f19:9c40:255b	ac:c1:ee:67:6a:6f	b8:76:3f:60:b3:c9	80	50502
18:40:13.187524	192.168.43.56	117.18.237.29	b8:76:3f:60:b3:c9	ac:c1:ee:67:6a:6f	57035	80
18:40:13.296466	117.18.237.29	192.168.43.56	ac:c1:ee:67:6a:6f	b8:76:3f:60:b3:c9	80	57035
18:40:17.299524			b8:76:3f:60:b3:c9	ac:c1:ee:67:6a:6f	58585	80
18:40:17.316499			ac:c1:ee:67:6a:6f	33:33:ff:32:e1:c1	80	58585
18:40:17.316830			ac:c1:ee:67:6a:6f	b8:76:3f:60:b3:c9	50502	80
18:40:17.392344			ac:c1:ee:67:6a:6f	b8:76:3f:60:b3:c9	80	50502
18:40:17.812032			ac:c1:ee:67:6a:6f	33:33:ff:32:e1:c1	57035	80
18:40:18.312077			ac:c1:ee:67:6a:6f	33:33:ff:32:e1:c1	80	57035
18:40:18.811109			ac:c1:ee:67:6a:6f	b8:76:3f:60:b3:c9	80	57035
18:40:20.292142			ac:c1:ee:67:6a:6f	33:33:ff:40:25:5b	58585	80
18:40:20.292169			b8:76:3f:60:b3:c9	ac:c1:ee:67:6a:6f	80	58585
18:40:20.931526	192.168.43.56	91.189.90.41	b8:76:3f:60:b3:c9	ac:c1:ee:67:6a:6f	50502	80

This page shows the details of each packet of ARP field as captured by tcpdump:

http://local...otocols.php x								
localhost/protocols.php								
Search								
---ARP PACKETS---								
TIMESTAMP	SOURCE IP	DESTINATION IP	SOURCE MAC	DESTINATION MAC	SOURCE PORT	DESTINATION PORT	SENDER LENGTH	RECIVER LENGTH
192.168.43	192.168.43	ac:c1:ee:67:6a:6f	ff:ff:ff:ff:ff:ff	18:40:02.902048	1	56	42	28
192.168.43	192.168.43	b8:76:3f:60:b3:c9	ac:c1:ee:67:6a:6f	18:40:08.051520	1	56	42	28
		ac:c1:ee:67:6a:6f	ff:ff:ff:ff:ff:ff	18:40:21.491743			42	28

TOOL/SOFTWARE USED

This project is based on L.A.M.P. Architecture. The LAMP refers to a solution stack of software, usually free and open source software, used to run dynamic Web sites or servers.

- Linux, the operating system
- Apache, the web server
- MySQL, the database
- PHP, the programming language.

LINUX OPERATING SYSTEM:

Linux, is a generic term referring to a family of Unix-like computer operating systems based on the Linux kernel. Their development is one of the most prominent examples of free and open source software collaboration; typically all the underlying source code can be used, freely modified, and redistributed by anyone. Users can control a Linux-based system through a command line interface (or CLI), a graphical user interface (or GUI), or through controls attached to the associated hardware (this is common for embedded systems). For desktop systems, the default mode is usually graphical user interface (or GUI).

A Linux system typically provides a CLI of some sort through a shell, which is the traditional way of interacting with a Unix system. A Linux distribution specialized for servers may use the CLI as its only interface. A “headless system” run without even a monitor can be controlled by the command line via a remote-control protocol such as SSH or telnet.

Advantages of using Linux:

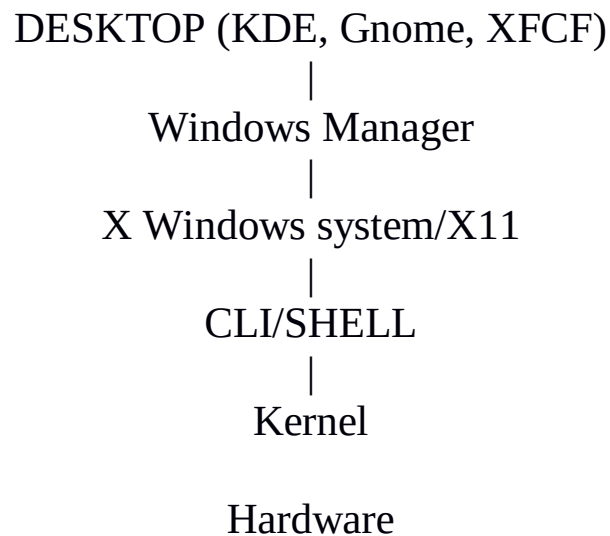
- **Upgradeability**
 1. The core of the Linux operating system is free and is uploaded constantly with new features and support for new hardware.
 2. Many of Linux’s large quantity of programs are “open source”, allowing the holder of the software to improve the software in whatever way they want as

long as credit is given.

- **Security**

1. Well-done administration of a computer with Linux allows for a very secure multi-user.
2. Workstation.
3. Virus is less of a threat

❖ **LINUX ARCHITECTURE:**



Linux system command is “tcpdump”.

❖ **APACHE HTTP SERVER:**

The **Apache HTTP Server**, commonly referred to as **Apache**, is a web server notable for playing a key role in the initial growth of the World Wide Web and in 2009 became the first web server to surpass the 100 million web site milestone .

Apache was the first viable alternative to the Netscape Communications Corporation web server (currently known as Sun Java System Web Server), and has since evolved to rival other Unix-based web servers in terms of functionality and performance. The majority of all web servers using Apache are Linux web servers.

Apache supports a variety of features, many implemented as compiled modules which extend the core functionality. These can range from server-side programming language support to authentication schemes. Some common language interfaces support mod_perl, mod_python, Tcl, and PHP. Popular authentication modules include mod_access, mod_auth, mod_digest, and mod_auth_digest, the successor to mod_digest.

Apache is primarily used to serve both static content and dynamic Web pages on the World Wide Web. Many web applications are designed expecting the environment and features that Apache provides.

Apache is the web server component of the popular LAMP web server application stack, alongside Linux, MySQL, and the PHP/Perl/Python (and now also Ruby) programming languages.

- **SHELL PROGRAMMING:** A SHELL is a command line interpreter which takes commands at the command line and then executes them. In general, a shell executes a programming language. The Bourne Shell in Linux can be used to create shell a script i.e. programmers that are interpreted and executed by the shell.

❖ MySQL:-

MYSQL is currently the most popular and also very useful open source database server than others, in existence. On top of that, MYSQL is very commonly used in conjunction with PHP scripts to create powerful and dynamic server-side applications and also database applications.

Features of MySQL:

- Stored procedures
- Triggers
- Cursors
- Updatable Views , Query caching
- True VARCHAR support
- INFORMATION_SCHEMA, SSL support

❖ PHP/HTML:-

PHP, which stands for "**Hypertext Preprocessor**", is a server-side, HTML embedded scripting language used to create Dynamic Web Pages. Much of its syntax is borrowed from C, Java and Perl with some unique features thrown in. The goal of the language is to allow Web developers to write dynamically generated pages quickly.

You can add that kind of functionality to your Web site by using PHP code. You will simply need an html form to get input from the user, and PHP code which will produce the intended feedback or record the information which the user entered.

RESULTS AND APPLICATION

The network monitoring tool was successfully developed. This tool can be an essential networking utility tool used by the centralized server system for any organization. The internet facility of any organization has to be monitored regularly so as to check for any cases of misuse and misapplication. So to keep a track of such things, all the data being transmitted from one terminal or system to another has to be recorded and analyzed. It can be used as a utility which can be installed on any server machine and thus is useful in finding the faults; misuse, hacking or any legal or illegal problem held at any attached terminal and can be removed in no time.

It gives us a tool, utility software which can give us solution to many networking problems in no time. With this only its objective area widens and so the scope. The end result of this project is that it will act as a utility tool and can be easily installed on any application server and there it can be analyzed as on what time which packet was communicating with the server, what data has being transferred, whether the application server is not being hacked or misused, and which of the terminals is unable to communicate with server due to crash of the system or any fault on any system terminal.

The scope also includes the help this project will provide to the users or the administrator. After being installed on any terminal it can be variety of checks can be done. Hacking check or some terminals fault check. To every illegal or any interrupted service occurs, alter is generated and is sent to the administrator to recover the faults. A search report is maintained and generated according to the search option.

FUTURE ASPECTS

While on a network, data being transmitted, including the addresses of the source and destination computers, is accessible to others who may be monitoring the network. For security, the data is often encrypted before being sent on the network. Along with the header and payload a 'cipher key' is passed along with each packet. The receiving end first decodes this cipher and then by use of cipher it decodes the data. Whole data is broken into several packets each of which contain header, cipher, payload, and footer.

In future work I need to extend the project so that it can decode the cipher and use that cipher to decode the data so that along with knowing infected IP using TOPTALKER we can also know what the infection is. Also telnet send data in plain text. But it is not in human readable form it still needs cipher to be decoded. In future we should be able to decode the data sent between two IPs in proper human readable format so that the centralized server has full power on network through this tool.

Conclusion

With the conclusion of this project, we can proudly claim that we've come a long way. We've seen how network analyzers are designed, how are they planned and how they are further implemented. We've also looked into the software technology that works in the background. Clearly this has been a revolutionary experience. We even had a chance to show off our learned skill.

In accordance with the project semester guidelines we can broadly classify the outcomes in the following points,

- Firstly, had a chance to understand and learn the basic and certain advanced concepts of networking and at the same time got hands-on practical skills.
- Learnt to break down a big problem into smaller ones and individually tackle those so as to solve one big problem.
- Plan out a system or approach to be taken and followed from the start to the end of the project.
- To be able to collect data and put it together into a concise and refined manner with appropriate tools.
- Have developed the ability to be able to consult experts in certain fields in the process of realization of this project.
- To be able to take decisions while tackling the problems keeping in mind the financial, environmental and social effects.
- While designing the network analyzer, able to identify advantageous tools and using them effectively.

APPENDIX A

PACKETS:

Data is sent/received across the Internet in chunks called packets. Before being transmitted over the network, the data (Which is nothing but stream of bits) is encapsulated at the various levels of the TCP/IP or OSI networking suite by appropriate headers and trailers.

These chunks of data along with the headers and trailers of each layer of the network protocol suite form what is known as the packets.

TCPDUMP:

TCPDUMP required ROOT ACCESS or the program must have suid of root. It is usually located in '/usr/sbin/tcpdump'.

The real magic of tcpdump is in its ability to focus. It can produce only the packets you're interested in, across a number of interfaces, protocols, destinations IP address (including specific ports), packet size, sessions, and so on. This magic comes in the form of a matching expression which can be entered at the command line.

SYNOPSIS:-

```
tcpdump [-aAbdDefIKlLnOpqRStuUvxX] [ -B size ] [ -c count ]
        [ -C file_size ] [ -E algo:secret ] [ -F file ] [ -G seconds ]
        [ -i interface ] [ -M secret ] [ -r file ]
        [ -s snaplen ] [ -T type ] [ -w file ] [ -W filecount ]
        [ -y datalinktype ] [ -z command ] [ -Z user ]
        [ expression ]
```

OPTIONS:-

-a Attempt to convert network and broadcast addresses to names.

- c Exit after receiving count packets.
- d Dump the compiled packet-matching code in a human readable form to standard output and stop.
- dd Dump packet-matching code as a C program fragment.
- ddd Dump packet-matching code as decimal numbers (pre-ceded with a count).
- e Print the link-level header on each dump line.
- f Print `foreign' internet addresses numerically rather than symbolically (this option is intended to get around serious brain damage in Sun's yp server -- usually it hangs forever translating non-local internet numbers).
- F Use file as input for the filter expression. An additional expression given on the command line is ignored.
- i listen on interface. If unspecified, tcpdump searches the system interface list for the lowest numbered, configured up interface (excluding loop- back). Ties are broken by choosing the earliest match.
- l Make stdout line buffered. Useful if you want to see the data while capturing it. E.g.,

```
``tcpdump -l | tee dat'' or
```

```
``tcpdump -l > Dat & tail -f dat''.
```
- n don't convert addresses (i.e., host addresses, port numbers, etc.) to names.
- N Don't print domain name qualification of host names. E.g., if you give this flag then tcpdump
Will print 'nic' instead of 'nic.ddn.mil'.
- O Do not run the packet-matching code optimizer. This is useful only if you suspect a bug in the Optimizer.

-p don't put the interface into promiscuous mode. Note that the interface might be in promiscuous mode for some other reason; hence, '-p' cannot be used as an abbreviation for 'ether host {local-hw-add} or ether Broadcast.

-q Quick (quiet?) output. Print less protocol information so output lines are shorter.

-r Read packets from file (which was created with the -w option). Standard input is used if file is '-'.

-T Force packets selected by "expression" to be interpreted the specified type. Currently known types are rpc (Remote Procedure Call), rtp (Real-Time Applications protocol), rtcp (Real-Time Applications control protocol), vat (Visual Audio Tool), and wb (distributed White Board).

-S Print absolute, rather than relative, TCP sequence numbers.

-t don't print a timestamp on each dump line.

-tt Print an unformatted timestamp on each dump line.

-v (Slightly more) verbose output. For example, the time to live and type of service information in an IP packet is printed.

-vv even more verbose output. For example, additional fields are printed from NFS reply packets.

-w Write the raw packets to file rather than parsing and printing them out. They can later be printed with the -r option. Standard output is used if file is '-'.

-x Print each packet (minus its link level header) in hex. The smaller of the entire packet or snaplen Bytes will be printed.

About last three lines of when tcpdump stops:

- packets "captured" (this is the number of packets that tcpdump has received

and processed)

- packets ``received by filter" (the meaning of this depends on the OS on which you're running tcpdump, and possibly on the way the OS was configured - if a filter was specified on the command line, on some OSes it counts packets regardless of whether they were matched by the filter expression and, even if they were matched by the filter expression, regardless of whether tcpdump has read and processed them yet, on other OSes it counts only packets that were matched by the filter expression regardless of whether tcpdump has read and processed them yet, and on other OSes it counts only packets that were matched by the filter expression and were processed by tcpdump);
- packets ``dropped by kernel" (this is the number of packets that were dropped, due to a lack of buffer space, by the packet capture mechanism in the OS on which tcpdump is running, if the OS reports that information to applications; if not, it will be reported as 0).

Time to Live:-

Time to live (sometimes abbreviated **TTL**) is a limit on the period of time or number of iterations or transmissions in computer and computer network technology that a unit data (e.g.a packet) can experience before it should be discarded.

The TTL field is set by the sender of the datagram, and reduced by every **host** on the route to its destination. If the TTL field reaches zero before the datagram arrives at its destination, then the datagram is discarded and an error datagram is sent back to sender. In IPV4 TTL is an 8-bit field in IP header.

The purpose of the TTL field is to avoid a situation in which an undeliverable datagram keeps circulating on an internet system, and such a system eventually becoming swamped by such immortal datagrams.

SEQUENCE NUMBER:

A fundamental notion in the design is that every octet of data sent over a TCP connection has a sequence number. Since every octet is sequenced, each of them can be acknowledged. The acknowledgment mechanism employed is cumulative so that an acknowledgment of sequence number X indicates that all octets up to but not including X have been received.

TRANSMISSION CONTROL PROTOCOL (TCP):

TCP provides reliable, in-order delivery of a stream of bytes, making it suitable for applications like file transfer and email. It is also responsible for controlling the size and rate at which messages are exchanged between the server and the client.

About Encryption and Decryption:

When a firewall computer receives a network packet from the network, it determines whether the transmission is authorized. If so, the computer examines the header within the packet to determine what encryption algorithm was used to encrypt the packet. Using this algorithm and a secret key, the computer decrypts the data and addresses of the source and destination computers and sends the data to the destination computer. If both the source and destination computers are firewall computers, the only addresses visible (i.e., unencrypted) on the network are those of the firewall computers. The addresses of computers on the internal networks, and, hence, the internal network topology, are hidden. This has been termed "virtual private networking" (VPN).

More about TCPDUMP -d Option:

As specified in manual -d option of TCPDUMP dump the compiled packets in human readable form and then stops, but it actually converts compiled code into assembly language that is more human readable in comparison to machine language code.

Load is denoted using **ldh**, Comparison is made using **jeq**, If true jumps shown by **jt**, if false jump shown by **jf**,

0x800 ether type of IPV4, **0x86dd** ether type of IPV6.

(007) ret #96 is equivalent to TRUE i.e. 96 bytes of packet data will be copied to calling application.

(008) ret #0 is equivalent of FALSE i.e. Zero bytes of packet data will be copied to applications.

APPENDIX B

Source codes of all relevant files

Html code for login page

```
<html>
<head>
<title>
"--NETWORK ANALYSER--"
</title>
<style
type="text/css">h1{border:double;color:black;background:red}body{font
:bold}</style>
</head>
<body bgcolor=blue ><center>
<h1>-NETWORK ANALYSER-</h1>
<form action="login.php" method="post">
<center>
<b>NAME:</b> <input type = "text" NAME="nm">
<br> <br>
<b>PASSWORD:</b> <input type= "password" name="pass">
<br>
<input type="submit" value="submit">
</center>
</form>
</body>
</html>
```

PHP code for connectivity

```
<?php
$myusername=isisset($_POST['nm'])? $_POST['nm']:"";
$mypassword=isisset($_POST['pass'])? $_POST['pass']:"";
$servername="localhost";

$conn=mysql_connect($servername,"root","");
if($conn==FALSE)
echo "connection failed";
//else
//echo "connected succesfully";
mysql_select_db("userlogin");
$sql = "select id from submit where username='$myusername' and password=
'$mypassword'";
    $result=mysql_query($sql,$conn);
    $count=mysql_num_rows($result);
if($count==1)
{
    system("/var/www/html/network.sh");
}
else
echo "your login name or password is invalid";
?>

<html>
<body bgcolor=76AFF8>
<head>
<center>
<b>
"---PLEASE SELECT YOUR PROTOCOL---"
</b>
</head>
<form action="protocols.php" method="POST">
<input type="radio" value="tcp" name="pro">
TCP
<br>
<input type="radio" value="udp" name="pro">
UDP
<br>
<input type="radio" value="arp" name="pro">
ARP
<br>
<input type="submit" value="submit" name="submit">

</center>
</form>
</body>
</html>
```

PHP code for protocol selection page

```
<?php
$result=isset($_POST['pro'])? $_POST['pro']:"";
if($result=="tcp")
{
    $file=fopen("tcpsip","r");
    $file1=fopen("tcpdip","r");
    $file2=fopen("tcpsmac","r");
    $file3=fopen("tcpdmac2","r");
    $file4=fopen("tcptimestamp","r");
    $file5=fopen("tcpsport","r");
    $file6=fopen("tcpdport","r");
    $file7=fopen("tcpslength","r");
    $file8=fopen("tcprlength","r");

    echo"<head><center><body bgcolor=93DC69><b>---TCP PACKETS---
</b></center></head><table border=10
bordercolor=blue><tr><th>TIMESTAMP</th><th>SOURCE IP</th><th>DESTINATION
IP</th><th>SOURCE MAC</th><th>DESTINATION MAC </th><th>SOURCE PORT</th>
<th> DESTINATION PORT</th><th>SENDER LENGTH</th><th>RECIVER
LENGTH</th></tr><tr><td>";
    while(!feof($file4)){echo fgets($file4)."<br>";};echo"</td><td>";
    while(!feof($file)){echo fgets($file)."<br>";};echo"</td><td>";
    while(!feof($file1)){echo fgets($file1)."<br>";};echo"</td><td>";
    while(!feof($file2)){echo fgets($file2)."<br>";};echo"</td><td>";
    while(!feof($file3)){echo fgets($file3)."<br>";};echo"</td><td>";
    while(!feof($file5)){echo fgets($file5)."<br>";};echo"</td><td>";
    while(!feof($file6)){echo fgets($file6)."<br>";};echo"</td><td>";
    while(!feof($file7)){echo fgets($file7)."<br>";};echo"</td><td>";
    while(!feof($file8)){echo
fgets($file8)."<br>";};echo"</td></tr></table></body>";
}

if($result=="udp")
{
    $file9=fopen("udpsip","r");
    $file10=fopen("udpdip","r");
    $file11=fopen("udpsmac","r");
    $file12=fopen("udpdmac2","r");
    $file13=fopen("udptimestamp","r");
    $file14=fopen("udpsport","r");
    $file15=fopen("udpdport","r");
    $file16=fopen("udpslength","r");
    $file17=fopen("udprlength","r");
```

```

    echo"<head><center><body><b>---UDP PACKETS---
</b></center></head><table border=10
bordercolor=blue><tr><th>TIMESTAMP</th><th>SOURCE IP</th><th>DESTINATION
IP</th><th>SOURCE MAC</th><th>DESTINATION MAC </th><th>SOURCE PORT</th>
<th> DESTINATION PORT</th><th>SENDER LENGTH</th><th>RECIVER
LENGTH</th></tr><tr><td>";

    while(!feof($file9)){echo fgets($file9)."<br>";};echo"</td><td>";
    while(!feof($file10)){echo fgets($file10)."<br>";};echo"</td><td>";
    while(!feof($file11)){echo fgets($file11)."<br>";};echo"</td><td>";
    while(!feof($file12)){echo fgets($file12)."<br>";};echo"</td><td>";
    while(!feof($file13)){echo fgets($file13)."<br>";};echo"</td><td>";
    while(!feof($file14)){echo fgets($file14)."<br>";};echo"</td><td>";
    while(!feof($file15)){echo fgets($file15)."<br>";};echo"</td><td>";
    while(!feof($file16)){echo fgets($file16)."<br>";};echo"</td><td>";
    while(!feof($file17)){echo
fgets($file17)."<br>";};echo"</td></tr></table></body>";

}

```

```

if($result=="arp")
{

```

```

    $file18=fopen("arpsip","r");
    $file19=fopen("arpdip","r");
    $file20=fopen("arpsmac","r");
    $file21=fopen("arpdmac2","r");
    $file22=fopen("arptimestamp","r");
    $file23=fopen("arpsport","r");
    $file24=fopen("arpdport","r");
    $file25=fopen("arpslength","r");
    $file26=fopen("arprlength","r");

```

```

    echo"<head><center><body><b>---ARP PACKETS---
</b></center></head><table border=10
bordercolor=blue><tr><th>TIMESTAMP</th><th>SOURCE IP</th><th>DESTINATION
IP</th><th>SOURCE MAC</th><th>DESTINATION MAC </th><th>SOURCE PORT</th>
<th> DESTINATION PORT</th><th>SENDER LENGTH</th><th>RECIVER
LENGTH</th></tr><tr><td>";

```

```

    while(!feof($file18)){echo fgets($file18)."<br>";};echo"</td><td>";
    while(!feof($file19)){echo fgets($file19)."<br>";};echo"</td><td>";
    while(!feof($file20)){echo fgets($file20)."<br>";};echo"</td><td>";
    while(!feof($file21)){echo fgets($file21)."<br>";};echo"</td><td>";
    while(!feof($file22)){echo fgets($file22)."<br>";};echo"</td><td>";
    while(!feof($file23)){echo fgets($file23)."<br>";};echo"</td><td>";
    while(!feof($file24)){echo fgets($file24)."<br>";};echo"</td><td>";
    while(!feof($file25)){echo fgets($file25)."<br>";};echo"</td><td>";

```

```

while(!feof($file26)){echo
fgets($file26)."<br>";};echo"</td></tr></table></body>";
}

```

?>

Shell script

```

#!/bin/sh
tcpdump -i wlan0 -ne -c 1000 > captured
grep "IP" captured > tcp
grep "UDP" captured > udp
grep "ARP" captured > arpextra
grep "Request" arpextra > arp

cut -d" " -f2 tcp > tcpsmac                #tcp source mac

cut -d" " -f4 tcp > tcpdmac1
sed 's/, //' tcpdmac1 > tcpdmac2            #destination mac

cut -d" " -f2 udp > udpsmac                #udp source mac

cut -d" " -f4 udp > udpdmac1
sed 's/, //' udpdmac1 > udpdmac2            #udp destination mac

cut -d" " -f2 arp > arpsmac                #arp source mac

cut -d" " -f4 arp > arpdmac1
sed 's/, //' arpdmac1 > arpdmac2            #for arp destiantion mac

cut -c-15 tcp > tcptimestamp               #tcp timestamp

cut -c-15 udp > udptimestamp               #udp timestamp

cut -c-15 arp > arptimestamp               #for arp timestamp

#-----port+ip-----

cut -d" " -f10 tcp > tcpsipplusp           #tcp source ip+port

cut -d" " -f12 tcp > tcpdipplusp           #tcp destination ip+port

cut -d" " -f10 udp > udpsipplusp           #udp source ip+port

cut -d" " -f12 udp > udpdipplusp           #udp destination ip+port

cut -d" " -f12 arp > arpsipplusp           #arp source ip+port

cut -d" " -f14 arp > arpdip1
sed 's/, $//' arpdip1 > arpdipplusp        #arp destiantion ip+port

#-----

```

```

cut -d" " -f9 arp > arpsleng #arp sender length
sed 's:///' arpsleng > arpslength
cut -d" " -f9 udp > udpsleng #udp sender length
sed 's:///' udpsleng > udpslength
cut -d" " -f9 tcp > tcpsleng #tcp sender length
sed 's:///' tcpsleng > tcpslength

grep -o ".....$" arp > arprleng #arp reciever length
cut -d" " -f2 arprleng > arprlength
grep -o ".....$" tcp > tcprleng #tcp reciever length
cut -d" " -f2 tcprleng > tcprlength
grep -o ".....$" udp > udprleng #udp reciever length
cut -d" " -f2 udprleng > udprlength

#-----

rev tcpsipplusp > tsip2
fgrep "." tsip2 > tsip23
cut -d"." -f1 tsip23 > tsip3
rev tsip3 > tcpSPORT #tcp source port

rev tcpdipplusp > tdip2
fgrep "." tdip2 > tdip23
cut -d"." -f1 tdip23 > tdip3
rev tdip3 > tdip4
sed 's:///' tdip4 > tcpdPORT #tcp destination port

sed 's/^[^.]*/:/' tsip2 > tsip7
rev tsip7 > tsip8
sed 's:///' tsip8 > tcpSIP #tcp source ip

sed 's/^[^.]*/:/' tdip2 > tdip7
rev tdip7 > tdip8
sed 's:///' tdip8 > tcpDIP #tcp destiantion ip

rev udpsipplusp > usip2
fgrep "." usip2 > usip23
cut -d"." -f1 usip23 > usip3
rev usip3 > udpsPORT #udp source port

rev udpdipplusp > udip2
fgrep "." udip2 > udip23
cut -d"." -f1 udip23 > udip3
rev udip3 > udip4
sed 's:///' udip4 > udpdPORT #udp destination port

sed 's/^[^.]*/:/' usip2 > usip7
rev usip7 > usip8
sed 's:///' usip8 > udpsIP #udp source ip

sed 's/^[^.]*/:/' udip2 > udip7
rev udip7 > udip8
sed 's:///' udip8 > udpdIP #udp destination ip

```



```

#-----
rev arpsipplusp > asip2
fgrep "." asip2 > asip23
cut -d"." -f1 asip23 > asip3
rev asip3 > arpsport                                #arp source port

rev arpdipplusp > adip2
fgrep "." adip2 > adip23
cut -d"." -f1 adip23 > adip3
rev adip3 > adip4
sed 's/:$//' adip4 > arpdport                        #arp destination port

sed 's/^[^.]*/:/' asip2 > asip7
rev asip7 > asip8
sed 's/.$//' asip8 > arpsip                          #arp source ip

sed 's/^[^.]*/:/' adip2 > adip7
rev adip7 > adip8
sed 's/.$//' adip8 > arpdip                          #arp destiantion ip

```

-----XXX-----